

Utilizando *funções*

Uma **função** é um bloco de código com nome, que você escreve uma vez e usa quantas vezes quiser. Funções evitam repetição, dão nome às ideias e deixam o programa fácil de testar. Nesta aula: `def`, chamar, parâmetros, `return`, a diferença entre `print` e `return`, escopo, docstring e dicas de tipo.

`def nome():`

Parâmetros

`return`

`print` × `return`

`None`

Escopo local e global

Docstring

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Defining Functions; Glossary: scope; PEP 257 docstrings), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Definir *e chamar*

definir.py

```
def saudar():  
    print("Olá!")  
    print("Bem-vindo ao curso.")  
  
saudar()          # chama a função  
saudar()          # e de novo
```

SAÍDA

```
Olá!  
Bem-vindo ao curso.  
Olá!  
Bem-vindo ao curso.
```

- ✓ `def nome():` com dois-pontos; o corpo vai com recuo
- ✓ Definir não executa nada: só ensina ao Python o que a função faz
- ✓ Chamar é `nome()`, com parênteses
- ✓ A função precisa estar definida **antes** da chamada
- ✓ Nome em snake_case, de preferência um verbo:
`calcular_total`

Parâmetros *e return*

parametro_retorno.py

```
def dobro(numero):  
    return numero * 2  
  
resultado = dobro(21)  
print(resultado)  
print(dobro(5) + dobro(10))
```

SAÍDA

```
42  
30
```

ARGUMENTO

21 vai na chamada



PARÂMETRO

vira `numero` lá dentro



RETURN

devolve 42 para quem chamou

- ✓ O valor devolvido pode ser guardado, impresso ou usado numa conta
- ✓ `dobro(5) + dobro(10)` : funções que devolvem valor se combinam

print *não é* return

print_vs_return.py

```
def soma_print(a, b):  
    print(a + b)  
  
def soma_return(a, b):  
    return a + b  
  
x = soma_print(2, 3)      # mostra 5, mas devolve None  
y = soma_return(2, 3)     # não mostra nada, devolve 5  
print("x =", x, "| y =", y)
```

SAÍDA

```
5  
x = None | y = 5
```

- ✓ `print` **mostra** na tela; o valor se perde
- ✓ `return` **devolve** o valor para o programa usar
- ✓ Função sem `return` devolve `None`

Regra prática: funções **calculam e devolvem**; quem mostra é o código que chama. Assim a mesma função serve para tela, arquivo ou teste.

return *encerra a função*

varios_retornos.py

```
def classificar(nota):  
    if nota >= 7:  
        return "aprovado"      # return encerra a função aqui  
    return "reprovado"  
  
for nota in [8.0, 5.5]:  
    print(nota, classificar(nota))
```

SAÍDA

```
8.0 aprovado  
5.5 reprovado
```

- ✓ Ao chegar num `return`, a função **termina na hora**
- ✓ Por isso não precisa de `else` depois de um `if` com `return`
- ✓ Para devolver vários valores, devolva uma tupla (aula 10):
`return menor, maior`

Variáveis *locais e globais*

escopo.py

```
taxa = 0.1          # variável global

def com_desconto(preco):
    desconto = preco * taxa    # local: só existe aqui dentro
    return preco - desconto

print(com_desconto(200))
print(taxa)
print(desconto)
```

```
180.0
0.1
Traceback (most recent call last):
  File "escopo.py", line 9, in <module>
    print(desconto)
    ^^^^^^^
NameError: name 'desconto' is not defined
```

- ✓ Variáveis criadas **dentro** da função (e os parâmetros) são **locais**: somem quando ela termina
- ✓ Por isso `desconto` dá **NameError** fora da função
- ✓ A função consegue ler variáveis globais (`taxa`)

Evite depender de globais: passe tudo o que a função precisa como parâmetro. Ela fica previsível e fácil de testar.

Docstring *e dicas de tipo*

docstring.py

```
def area_retangulo(base: float, altura: float) -> float:
    """Calcula a área de um retângulo."""
    return base * altura

print(area_retangulo(3, 4.5))
print(area_retangulo.__doc__)
```

SAÍDA

```
13.5
Calcula a área de um retângulo.
```

- ✓ `"""..."""` logo abaixo do `def` é a **docstring**: o Cursor mostra ao passar o mouse
- ✓ `base: float` e `-> float` são **dicas de tipo**: documentam, não obrigam (capítulo 17 do guia)
- ✓ `help(area_retangulo)` no REPL mostra a documentação

Dica do guia (cap. 7): funções devem fazer **uma coisa** e fazer bem. Passou de 20 linhas, considere dividir.

Refatorando *em funções*

Neste arquivo, a conta do desconto aparece repetida em três lugares. Me mostre, sem editar, como ficaria com uma função `calcular_desconto:` nome, parâmetros, o que ela devolve e onde chamar. Eu mesmo faço a mudança.

Escreva 3 testes simples (com `assert`) para a minha função `media(notas)`, incluindo uma lista com uma nota só.

- ✓ Código repetido é o melhor sinal de que falta uma função
- ✓ Peça o **desenho** da função e implemente você
- ✓ Funções pequenas que devolvem valor são fáceis de testar com `assert` ou `pytest` (capítulo 20 do guia)

Funções *que se combinam*

Enunciado: crie `media(notas)` e `situacao(valor)` (aprovado a partir de 7, senão recuperação) e use as duas para mostrar o resultado de uma lista de notas.

Uma resposta possível

```
# Exercício: funções pequenas que se combinam
def media(notas):
    return sum(notas) / len(notas)

def situacao(valor):
    return "aprovado" if valor >= 7 else "recuperação"

notas = [7.5, 8.0, 6.5]
m = media(notas)
print(f"Média {m:.2f}: {situacao(m)}")
```

SAÍDA

Média 7.33: aprovado

- ✓ Cada função faz uma coisa: calcular; classificar
- ✓ Nenhuma das duas usa `print`: quem mostra é o código de fora
- ✓ Desafio: o que acontece com `media([])`? Trate esse caso

Teste o que *ficou*

1. O que uma função sem return devolve?

A) 0

B) None

C) ""

D) Erro

B.

2. Variável criada dentro da função, usada fora:

A) Funciona

B) NameError

C) Vale None

D) Vale 0

B. É local.

3. O que acontece ao executar um return?

A) Imprime o valor

B) A função termina e devolve o valor

C) Repete a função

D) Nada

B.

4. def f(x): ... — o x é:

A) Um argumento

B) Um parâmetro

C) Um retorno

D) Uma global

B. Na chamada f(5), o 5 é o argumento.

Escreva uma vez, *use sempre*

`def` cria a função, a chamada a executa. Parâmetros recebem os argumentos; `return` devolve um valor e encerra a função (sem ele, `None`). Variáveis de dentro são locais. Docstring e dicas de tipo documentam, e funções pequenas que devolvem valor são as mais fáceis de reusar e testar.

`def` · chamada

`return`

`print` ≠ `return`

Local × global

Docstring

Próxima aula → 18 · Argumentos em Funções

Fontes: docs.python.org/3/tutorial/controlflow.html#defining-functions; docs.python.org/3/glossary.html (parameter, argument); peps.python.org/pep-0257, consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.